Overview of the Authentication Flow

When a user attempts to log in, Spring Security follows a specific internal flow to verify credentials against a database:

1. **Filter:** UsernamePasswordAuthenticationFilter extracts credentials.
2. **Authentication Provider:** DaoAuthenticationProvider is responsible for fetching user details.
3. **UserDetailsService:** A custom service that talks to the Repository to find the user in the DB.
4. **UserDetails:** A wrapper (Principal) that converts your DB Entity into a format Spring Security understands.

Step 1: Dependencies (pom.xml)

To connect to a database and use JPA, you need the following dependencies in your Maven project:

XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

Step 2: Database Configuration

SQL Table Setup

In your PostgreSQL (PGAdmin), create a table named users. Using the name users is often safer than user as "user" is a reserved keyword in many SQL dialects.

SQL

```sql
CREATE TABLE users (
    id INT PRIMARY KEY,
    username TEXT NOT NULL,
    password TEXT NOT NULL
);

INSERT INTO users VALUES (1, 'naven', 'n123'), (2, 'sujal', 's123');
```

Application Properties

Add the connection details in src/main/resources/application.properties:

Properties

```properties
spring.datasource.url=jdbc:postgresql://localhost:5432/telusko_db
spring.datasource.username=postgres
spring.datasource.password=0000
spring.jpa.hibernate.ddl-auto=update
```

Step 3: Creating the Model & Principal

The Entity Class

This class maps directly to your database table.

Java

```java
@Entity
@Table(name = "users")
public class User {
    @Id
    private int id;
    private String username;
    private String password;

    // Getters and Setters
}
```

The UserPrincipal (UserDetails)

Spring Security cannot use your User entity directly. You must create a wrapper that implements UserDetails.

Java

```java
public class UserPrincipal implements UserDetails {
    private User user;

    public UserPrincipal(User user) {
        this.user = user;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        // Returning a hardcoded "USER" role for now
        return Collections.singleton(new SimpleGrantedAuthority("USER"));
    }

    @Override
    public String getPassword() { return user.getPassword(); }

    @Override
    public String getUsername() { return user.getUsername(); }

    @Override
    public boolean isAccountNonExpired() { return true; }

    @Override
    public boolean isAccountNonLocked() { return true; }

    @Override
    public boolean isCredentialsNonExpired() { return true; }

    @Override
    public boolean isEnabled() { return true; }
}
```

Step 4: Repository & Custom UserDetailsService

The Repository

Java

```java
@Repository
public interface UserRepository extends JpaRepository<User, Integer> {
    User findByUsername(String username);
}
```

The Service

This service is called by Spring Security to load the user data.

Java

```java
@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = repo.findByUsername(username);
        if (user == null) {
            System.out.println("User Not Found");
            throw new UsernameNotFoundException("User not found");
        }
        return new UserPrincipal(user);
    }
}
```

Step 5: Security Configuration

Finally, wire everything together in your SecurityConfig class by defining an AuthenticationProvider.

Java

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private UserDetailsService userDetailsService;

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();

        // Disable password encoding for testing purposes (NoOp)
        provider.setPasswordEncoder(NoOpPasswordEncoder.getInstance());
        provider.setUserDetailsService(userDetailsService);

        return provider;
    }
}
```

Output Examples

1. Successful Login

Request (Postman):

- **Method:** GET http://localhost:8080/students
- **Auth:** Basic Auth (Username: naven, Password: n123)

Output:

- **Status:** 200 OK
- **Body:** [{"id": 1, "name": "Dilshan", "marks": 85}, ...]

2. Failed Login (Wrong Password)

Request:

- **Auth:** Basic Auth (Username: naven, Password: wrong_password)

Output:

- **Status:** 401 Unauthorized
- **Body:** ```json
  {

"timestamp": "2026-04-01...",
"status": 401,
"error": "Unauthorized"
}

3. Failed Login (User Not Found)

If you try to log in with a user not in the database (e.g., unknown_user):

- **Console Output:** User Not Found (from our custom print statement).
- **HTTP Status:** 401 Unauthorized.

What can go wrong?

Issue	Cause	Likely Error/Output
Connection Refused	DB Server is not running or wrong Port/URL.	JDBCConnectionException on startup.
Table not found	Table is named user (reserved) or doesn't exist.	PSQLException: relation "users" does not exist.
Infinite Loop/Recursion	Forgetting to implement UserDetails correctly.	StackOverflowError or Beans not loading.
Field Mismatch	DB column is user_name but Java field is username.	PropertyReferenceException or Null values.

Does this structure help you visualize how the different layers (Service, Repo, and Config) interact to secure your application?

ඔයාට හිතෙන්න පුළුවන් මේක නිකන් →වටේ යන වැඩක්→ වගේ කියලා. හැබැයි Spring Security වල මේ විදිහට `UserDetails`, `UserDetailsService`, සහ `DaoAuthenticationProvider` පාවිච්චි කරන්නේ ප්‍රධාන හේතු කීපයක් නිසා. මේක **Separation of Concerns** (වගකීම් බෙදා වෙන් කිරීම) කියන concept එක උඩ තමයි හැදිලා තියෙන්නේ.

ඇයි කෙලින්ම Database එකෙන් ගන්න බැරි සහ මේ layers වල අවශ්‍යතාවය මෙන්න මෙහෙමයි:

1. Framework එකට Database එක ගැන දැනුමක් නැති වීම

Spring Security කියන්නේ ඕනෑම database එකක් (MySQL, PostgreSQL, MongoDB, LDAP) එක්ක වැඩ කරන්න පුළුවන් framework එකක්. ඔයාගේ Database එකේ තියෙන `User` entity එකේ fields (උදාහරණ: `address`, `date_of_birth`) framework එක දන්නේ නැහැ.

- **විසඳුම:** ඔයාගේ DB එකේ තියෙන user data ටික Spring Security ට තේරෙන →Common Format→ එකකට හරවන්න ඕනේ. ඒ format එක තමයි **`UserDetails`**.

2. `UserDetailsService` (The Data Fetcher)

මේක නිකන් →කෑම බෙදන කෙනෙක්→ වගේ. Spring Security කියනවා →මට අහවල් username එක තියෙන user ව හොයලා දෙන්න→ කියලා. ඒ user ඉන්නේ Database එකේද, නැත්නම් වෙනත් API එකකද කියලා framework එකට වැඩක් නැහැ.

- ඔයා කරන්නේ `loadUserByUsername` method එක ඇතුලේ **Custom Logic** එක ලියන එකයි (උදාහරණ: `repo.findByUsername()`). මේක නිසා ඔයාට ඕනෑම තැනකින් user ව fetch කරලා දීමේ නිදහස ලැබෙනවා.

3. `DaoAuthenticationProvider` (The Logical Brain)

User ව database එකෙන් හොයාගත්තට පස්සේ තව ගොඩක් දේවල් කරන්න තියෙනවා:

- User ගේ password එක match වෙනවද? (Password Hashing/Encoding)
- Account එක lock වෙලාද?
- Account එක expired වෙලාද?
- User ට තියෙන Roles මොනවාද?

ඔයා මේ දේවල් හැම එකම manually හැම project එකකම ලියන්න ගියොත් ඒක වෙලාව නාස්ති වීමක් සහ වැරදි වෙන්න තියෙන ඉඩ කඩ වැඩියි. **`DaoAuthenticationProvider`** එක කරන්නේ අර `UserDetails` object එක අරගෙන මේ logic ටික (Validation) ඔක්කොම automatic කරලා දෙන එකයි.

Architecture එක සරලව

1. **Filter:** User එවපු username/password අල්ලගන්නවා.
2. **Provider (`DaoAuthenticationProvider`):** →මම මේක check කරන්නම්→ කියලා ඉදිරිපත් වෙනවා.
3. **Service (`UserDetailsService`):** Provider කියනවා →මට database එකෙන් මේ user ව හොයලා දෙන්න→ කියලා.
4. **UserDetails:** Service එක database එකෙන් user ව අරගෙන standard format එකකට (UserDetails) දාලා ආපහු දෙනවා.
5. **Validation:** Provider එක password check කරලා ඔක්කොම හරි නම් login එක success කරනවා.

සාරාංශය

ඔයාට කෙලින්ම Database එකෙන් ගන්න බැරි නැහැ, හැබැයි එහෙම කළොත් Spring Security වල තියෙන **Password Encoding, Account Locking, සහ Role-based Authorization** වගේ features ඔයාට manually ලියන්න වෙනවා. මේ layers පාවිච්චි කරාම වැඩේ safe වගේම ලේසියි.

මේ concept එකේ `Password Encoder` එක වැඩ කරන විදිහ ගැනත් ඔයාට දැනගන්න ඕනෙද?

මේක මුලින්ම බලද්දී පොඩ්ඩක් සංකීර්ණයි වගේ පෙනුනට, මේකේ තියෙන්නේ හරිම ලස්සන flow එකක්. අපි මේ කරපු සම්පූර්ණ process එක පියවරෙන් පියවර සරලව තේරුම් ගමු.

1. ඇයි `UserPrincipal` එකක් හැදුවේ?

ඔයාගේ Database එකේ තියෙන `User` class එක (Entity එක) අයිති ඔයාට. ඒකේ `id`, `username`, `password` විතරක් නෙවෙයි, `email`, `address` වගේ ඕනෑම field එකක් තියෙන්න පුළුවන්. හැබැයි **Spring Security** ඔයාගේ class එක ගැන කිසිම දෙයක් දන්නේ නැහැ.

Spring Security ට ඕනේ කරන්නේ එයාට වැඩ කරන්න පුළුවන් →Standard Object→ එකක්. ඒක තමයි **`UserDetails`** කියන interface එක.

- **UserPrincipal** කියන්නේ →Wrapper→ එකක්.
- මේකෙන් කරන්නේ අපේ `User` object එක අරගෙන, ඒක Spring Security ට තේරෙන `UserDetails` format එකට →Wrap→ කරලා දෙන එකයි.
- එතකොට Spring Security ට පුළුවන් `getUsername()`, `getPassword()` වගේ methods

පාවිච්චි කරලා ඔයාගේ object එක ඇතුලේ තියෙන data කියවන්න.

2. සම්පූර්ණ සිද්ධිය මුල සිට (Step-by-Step)

මෙන්න මේකයි අපි කරපු දේ සහ ඒක වුනු විදිහ:

Step A: Database එක ලෑස්ති කිරීම (Setup)

මුලින්ම අපි `pom.xml` එකට **Spring Data JPA** සහ **PostgreSQL Driver** එක එකතු කළා. ඊට පස්සේ `application.properties` එකේ DB URL එක සහ credentials දුන්නා. මේකෙන් තමයි Spring Boot වලට Database එකත් එක්ක කතා කරන්න පාර හැදුනේ.

Step B: Data නියෝජනය කිරීම (Model & Repo)

- **Entity Class (`User`):** Database table එකේ තියෙන rows ටික Java object එකක් විදිහට map කරගන්න මේක පාවිච්චි කළා.
- **Repository (`UserRepository`):** Database එකෙන් data select කරලා ගන්න (SQL queries ලියන්නේ නැතුව) මේ interface එක හැදුවා. මෙතනදී අපි `findByUsername` කියන custom method එකත් හැදුවා.

Step C: Security Wrapper එක හැදීම (`UserPrincipal`)

අපි කලින් කතා කරපු විදිහට, අපේ `User` object එක Spring Security ට තේරෙන විදිහට සකස් කරලා දුන්නේ මේ class එකෙන්. මෙතනදී තමයි user ගේ roles (Authorities) මොනවාද, account එක lock වෙලාද වගේ දේවල් check කරන්නේ.

Step D: User ව Fetch කිරීම (`MyUserDetailsService`)

මේක තමයි ලොකුම වැඩේ කරන්නේ. User කෙනෙක් login වෙන්න හදනකොට, Spring Security මේ service එකට කියනවා →**මෙන්න මේ username එක තියෙන user ව මට හොයලා දෙන්න**→ කියලා.

- එතකොට මේ class එක අර `UserRepository` එක පාවිච්චි කරලා DB එකෙන් user ව හොයනවා.
- User ව හම්බුනොත්, එයාව `UserPrincipal` එකකට දාලා ආපහු Spring Security ට දෙනවා.

Step E: Security Config එක සෙට් කිරීම (`SecurityConfig`)

අන්තිමට අපි Spring Security ට කිව්වා →**දැන් default එක පාවිච්චි කරන්න එපා, මම හදපු `DaoAuthenticationProvider` එක පාවිච්චි කරන්න**→ කියලා.

- මේ `DaoAuthenticationProvider` එකට අපි අපේ `MyUserDetailsService` එක සම්බන්ධ කළා.
- දැන් user password එකක් ගැහුවම, Provider එක කරන්නේ Service එකෙන් user ව ගෙන්නගෙන, password එක match වෙනවද කියලා check කරන එකයි.

සාරාංශය: මේකේ වාසිය මොකක්ද?

පියවර	වගකීම
Repository	Database එකෙන් raw data අරන් දෙනවා.
UserDetailsService	Username එක අනුව user ව හොයලා `UserDetails` එකක් හදනවා.
UserDetails (Principal)	User ගේ දත්ත Spring Security ට තේරෙන භාෂාවට හරවනවා.
AuthenticationProvider	Password එක හරිද කියලා අවසාන තීරණය ගන්නවා.

දැන් ඔයාට පැහැදිලිද ඇයි අපි කෙලින්ම DB එකෙන් data ගන්නේ නැතුව මේ වගේ →Layers→ පාවිච්චි කරන්නේ කියලා? මේකෙන් අපිට ලැබෙන ලොකුම වාසිය තමයි අපේ code එක ඉතාමත් **Flexible** (නම්‍යශීලී) වීම.

මේ process එකේදී code එකේ කොහේ හරි තැනක් තවදුරටත් පැහැදිලි වෙන්න ඕනෙද? උදාහරණයක් විදිහට `NoOpPasswordEncoder` එක ඇයි පාවිච්චි කළේ වගේ දෙයක්?

ඔව්, **`DaoAuthenticationProvider`** කියන්නේ Spring Security වල තියෙන එකම `AuthenticationProvider` එක නෙවෙයි. Spring Security නිර්මාණය කරලා තියෙන්නේ ඉතාමත්

modular විදිහට, ඒ නිසා ඔයාගේ අවශ්‍යතාවය අනුව පාවිච්චි කරන්න පුළුවන් තවත් providers කිහිපයක් තියෙනවා.

මෙන්න ප්‍රධාන විකල්ප කිහිපයක්:

1. Custom AuthenticationProvider (වඩාත්ම ජනප්‍රිය විකල්පය)

ඔයාට `UserDetailsService` සහ `UserDetails` වගේ standard flow එක පාවිච්චි කරන්නේ නැතුව, සම්පූර්ණයෙන්ම ඔයාටම ආවේණික (Custom) විදිහකට authentication කරන්න ඕනේ නම්, ඔයාට පුළුවන් කෙලින්ම **`AuthenticationProvider`** interface එක implement කරලා අලුත් class එකක් හදන්න.

- **ඇයි මේක කරන්නේ?** උදාහරණයක් විදිහට, ඔයාට user ව verify කරන්න ඕනේ Database එකකින් නෙවෙයි, වෙනත් External REST API එකකට call එකක් දීලා නම්, මේක පාවිච්චි කරන්න පුළුවන්.
- **විශේෂත්වය:** මෙතනදී `UserDetailsService` අවශ්‍ය වෙන්නේ නැහැ. ඔයාට කැමති logic එකක් `authenticate()` method එක ඇතුලේ ලියන්න පුළුවන්.

2. LdapAuthenticationProvider

ලොකු companies වල පාවිච්චි කරන **Active Directory** හෝ **LDAP** (Lightweight Directory Access Protocol) server එකක් හරහා user ව login කරවන්න ඕනේ නම් මේ provider එක පාවිච්චි කරනවා.

- මෙතනදී user data තියෙන්නේ සාමාන්‍ය database එකක නෙවෙයි, directory service එකක.

3. OAuth2LoginAuthenticationProvider

ඔයා →Login with Google→ හෝ →Login with GitHub→ වගේ options app එකට දානවා නම් පාවිච්චි වෙන්නේ මේකයි. මේකෙදි අපේ database එකේ password check වෙන්නේ නැහැ, ඒ වෙනුවට Google/GitHub වගේ **Third-party Identity Provider** කෙනෙක් තමයි user ව verify කරලා දෙන්නේ.

4. RememberMeAuthenticationProvider

User කෙනෙක් login වෙද්දී →Remember Me→ කියන tick එක දැම්මම, ඊළඟ පාර session එක expired වුණත් ආපහු login වෙන්න අවශ්‍ය වෙන්නේ නැහැ. ඒ functionality එක handle කරන්නේ මේ provider එකෙන්.

සංසන්දනය: ඇයි DaoAuthenticationProvider තෝරාගන්නේ?

Provider	පාවිච්චි කරන අවස්ථාව
DaoAuthenticationProvider	Database එකක තියෙන Username/Password check කරන සාමාන්‍ය app එකකට.
LdapAuthenticationProvider	Corporate/Office networks වල තියෙන directory services වලට.
Custom Provider	ඉතාමත් සංකීර්ණ logic එකක් (උදා: External API call, Multi-factor auth) තියෙනවා නම්.
OAuth2 Provider	Social Logins (Google, Facebook) පාවිච්චි කරනවා නම්.

ඔයාට හිතෙනවා නම් UserDetailsService වටේ යන වැඩක් කියලා...

ඔයාට පුළුවන් කෙලින්ම **Custom AuthenticationProvider** එකක් හදන්න. එතකොට DaoAuthenticationProvider එක අයින් කරලා, ඔයා ලියන logic එකෙන්ම database එකට කතා කරලා user ව verify කරන්න පුළුවන්.

ඔයාට ඒ වගේ **Custom AuthenticationProvider** එකක් හදන විදිහ ගැන code example එකක් අවශ්‍යද? එහෙම නැත්නම් මේ flow එකේ තව මොකක් හරි පැහැදිලි නැති තැනක් තියෙනවද?

ඔයා කලින් කරපු Microservice project එකේදී CRUD operations කළාට මේ වගේ Security configuration එකක් හමු නොවුණේ හේතු කිහිපයක් නිසා වෙන්න පුළුවන්:

1. **CRUD vs Authentication:** CRUD (Create, Read, Update, Delete) කියන්නේ user කෙනෙක්ගේ data database එකේ save කරන එක විතරයි. හැබැයි **Security** කියන්නේ

ඒ user ගැහුව password එක හරිද කියලා check කරලා, එයාට system එක ඇතුළට එන්න අවසර දෙන (Authentication) එකයි.

2. **Microservices Gateway:** ගොඩක් වෙලාවට microservices වලදී security handle කරන්නේ **API Gateway** එකෙන් (උදා: JWT tokens පාවිච්චි කරලා). එතකොට අනිත් microservices වල මේ වගේ configuration ලියන්න අවශ්‍ය වෙන්නේ නැහැ.
3. **Default Security:** සමහරවිට ඔයා Spring Security dependency එක add කරලා නොතිබෙන්න ඇති, එහෙම නැත්නම් default settings ම පාවිච්චි වෙන්න ඇති.

දැන් අපි බලමු **Custom AuthenticationProvider** එකක් හදන්නේ කොහොමද කියලා. මේකේ විශේෂත්වය තමයි මෙතැනදී අපිට `UserDetailsService` හෝ `UserDetails` කියන →වටේ යන→ classes ඕන වෙන්නේ නැහැ. කෙලින්ම repository එකෙන් user ව අරන් check කරන්න පුළුවන්.

Custom AuthenticationProvider Example

මෙන්න මේ class එක හදලා ඔයාගේ Repository එක කෙලින්ම මේකට inject කරන්න.

Java

```
@Component

public class MyCustomAuthProvider implements AuthenticationProvider {



    @Autowired

    private UserRepository repo; // Directly using your JPA Repository



    @Override

    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
```

```
        // Get username and password from the login request

        String username = authentication.getName();

        String password = authentication.getCredentials().toString();



        // 1. Fetch user from DB directly using Repository

        User user = repo.findByUsername(username);



        // 2. Validate user and password

        if (user != null && user.getPassword().equals(password)) {



            // If successful, return an Authentication object (UsernamePasswordAuthenticationToken)

            // This tells Spring Security: "This user is valid!"

            return new UsernamePasswordAuthenticationToken(username, password, new
ArrayList<>());

        } else {

            // If fails, throw an exception

            throw new BadCredentialsException("Invalid Username or Password!");

        }
```

```
    }



    @Override

    public boolean supports(Class<?> authentication) {

        // This provider supports Username/Password style authentication

        return authentication.equals(UsernamePasswordAuthenticationToken.class);

    }

}
```

Security Configuration එකට මේක සම්බන්ධ කරන්නේ මෙහෙමයි:

ඔයාගේ `SecurityConfig` class එක ඇතුලේ මේ provider එක register කරන්න ඕනේ.

Java

```
@Configuration

@EnableWebSecurity

public class SecurityConfig {



    @Autowired

    private MyCustomAuthProvider customAuthProvider;
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(customizer -> customizer.disable())
                .authorizeHttpRequests(request -> request.anyRequest().authenticated())
                .httpBasic(Customizer.withDefaults())
                .authenticationProvider(customAuthProvider) // Registering our custom provider
                .build();
    }
}
```

මොකක්ද මෙතන වුණේ? (The Flow)

1. User ලොගින් වෙන්න හදනකොට Spring Security එකට Username/Password ලැබෙනවා.
2. Spring Security එක අපේ `MyCustomAuthProvider` එකේ තියෙන `authenticate()` method එකට ඒ data එවනවා.
3. අපි ඒ method එක ඇතුළේ කෙලින්ම **`repo.findByUsername()`** කියලා database එකෙන් user ව හොයනවා.
4. අපි password එක manually check කරනවා (`user.getPassword().equals(password)`).
5. Password එක හරි නම්, අපි `UsernamePasswordAuthenticationToken` එකක් return කරනවා. ඒකෙන් තමයි Spring Security එක දැනගන්නේ user →Verified→ කියලා.

මේ ක්‍රමයේ තියෙන වෙනස:

- **පහසුව:** `UserDetailsService` සහ `UserPrincipal` වගේ ගොඩක් classes හදන්න ඕන වෙන්නේ නැහැ.
- **පාලනය:** Authentication logic එක (උදා: user ට email verification එකක් ඇවිල්ලද කියලා බලන එක වගේ දේවල්) සම්පූර්ණයෙන්ම ඔයාට කැමති විදිහට එකම තැනක ලියන්න පුළුවන්.

හැබැයි මතක තියාගන්න, real-world apps වලදී අපි කවදාවත් මෙහෙම `equals()` දාලා password check කරන්නේ නැහැ. අනිවාර්යයෙන්ම **BCrypt** වගේ password encoder එකක් පාවිච්චි කරන්න ඕනේ.

දැන් ඔයාට CRUD සහ Security අතර වෙනස පැහැදිලිද? මීළඟට අපිට මේ passwords ටික encrypt කරලා database එකේ save කරන්නේ කොහොමද කියලා බලන්න පුළුවන්. ඒ ගැන දැනගන්න කැමතිද?